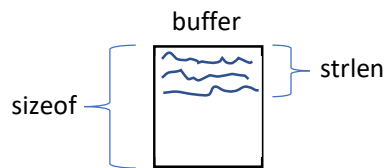


TCP

atoi() → converte um array de caracteres (string) num inteiro

strcmp() → compara 2 arrays de caracteres (strings). Retorna 0 (zero) se forem iguais



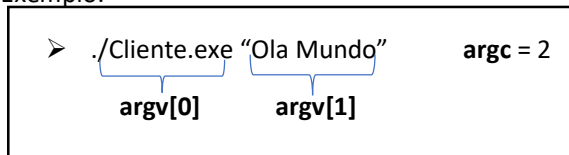
sizeof() → retorna o tamanho total do buffer

strlen() → retorna o comprimento de um array de caracteres

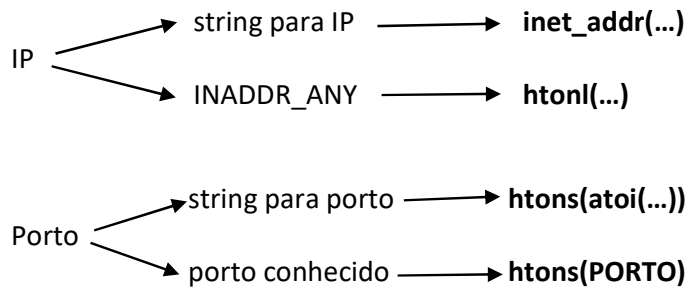
argc → indica o número de argumentos que foram passados na linha de comandos

argv → array de argumentos que foram passados na linha de comandos

Exemplo:



Registrar:



Mostrar

IP → `inet_ntoa(...)`

Porto → `ntohs(...)`

inet_addr() → converte uma string no formato *dotted decimal* (exemplo: 127.0.0.1) para um endereço IP que será reconhecido pela rede TCP/IP.

htonl() → host to network long. De modo básico, estamos a converter um endereço de host para algo que será reconhecido pela rede TCP/IP.

htons() → host to network short. De modo básico, estamos a converter um número em algo que será reconhecido pela rede TCP/IP.

ntohs() → network to host short. De modo básico, estamos a converter um porto reconhecido pela rede TCP/IP para um número.

inet_ntoa() → converte um endereço IP para algo que será reconhecido por nós (string no formato dotted decimal)

TCP

- Orientado a fluxo de dados (ponto a ponto)
 - Fiável: sem erros, dados ordenados, não duplicados
 - Orientado a ligação (estabelece uma ligação antes de transmitir os dados): não suporta broadcast
-

Passos básicos do cliente e servidor

Cliente

Iniciar Winsocks

Abrir socket

Preencher endereço
servidor

Estabelecer ligação

Enviar dados

Receber dados

Fechar socket

Servidor

Iniciar Winsocks

Abrir socket para escuta

Preencher endereço
escuta

Associar socket ao
endereço de escuta

Avisar que está pronto
para aceitar pedidos

Aceitar cliente

Receber dados

Enviar dados

Fechar socket

Abrir socket para escuta

```
1      2      3  
sock = socket(PF_INET, SOCK_STREAM, IPPROTO_TCP);  
if (sock == INVALID_SOCKET)  
    Abort("Impossibilidade de abrir socket", sock);
```

1. Especificação da família de endereços. No nosso caso vamos usar endereços no formato IPv4, designados por **PF_INET** (ou AF_INET).
2. Tipo do socket, neste caso socket para stream de dados.
3. Protocolo a ser usado, neste caso TCP.

Nota: em alguns exercícios podem ver este parâmetro com o valor 0 (zero). A função *socket* permite que este valor seja 0, e nesse caso, não estamos a especificar diretamente que queremos usar o protocolo TCP. O protocolo a usar será definido pelo *service provider*.

Fonte: <https://learn.microsoft.com/en-us/windows/win32/api/winsock2/nf-winsock2-socket>

Preencher endereço

```
1 memset((char*)&serv_addr, 0, sizeof(serv_addr));  
2 serv_addr.sin_family = AF_INET;  
3 serv_addr.sin_addr.s_addr = inet_addr(SERV_HOST_ADDR) OU htonl(INADDR_ANY);  
4 serv_addr.sin_port = htons(SERV_UDP_PORT);
```

1. Coloca a 0 (zero) todos os bytes da estrutura *serv_addr*
2. Família de endereços (Internet)
3. Endereço do host

Cliente: IP do endereço do servidor (usando a função *inet_addr*)

Servidor: ao usar o *INADDR_ANY* (com a função *htonl*) estamos a dizer que estamos à escuta de stream/fluxo de dados vindos de qualquer endereço de rede (da família especificada no ponto 2)

4. Porto que identifica a aplicação

Nota: Tenham atenção às funções *inet_addr()*, *htonl()*, *htons()*,...

Associar socket ao endereço de escuta

```
int result;  
result = bind(sock, (struct sockaddr*)&serv_addr, sizeof(serv_addr));  
if (result == SOCKET_ERROR)  
    Abort("Impossibilidade de registar-se para escuta", sock);
```

1. Sock (criado anteriormente)
 2. Endereço da estrutura que contém os dados do servidor (família de endereços, IP e Porto)
 3. Tamanho da estrutura definida no ponto 2
-

Avisar que está pronto a aceitar pedidos

```
int result;  
result = listen(sock, 5);  
if (result == SOCKET_ERROR)  
    Abort("Impossibilidade de escutar pedidos", sock);
```

1. Sock (criado anteriormente)
2. Número máximo de ligações em espera

Estabelecer ligação

```
int result;  
result = connect(sock, (struct sockaddr*)&serv_addr, sizeof(serv_addr));  
if (result == SOCKET_ERROR)  
    Abort("Impossibilidade de estabelecer ligacao", sock);
```

1. Sock (criado anteriormente)
 2. Endereço da estrutura que contém os dados (família de endereços, IP e Porto) à qual nos queremos conectar
 3. Tamanho da estrutura que contém os dados do endereço que nos queremos conectar
-

Aceitar cliente

```
SOCKET newSock;  
int cliaddr_len;  
  
cliaddr_len = sizeof(cli_addr);  
  
newSock = accept(sock, (struct sockaddr*)&cli_addr, &cliaddr_len);  
if (newSock == SOCKET_ERROR)  
    fprintf(stderr, "<SERV> Impossibilidade de aceitar cliente...\n");
```

1. Sock (criado anteriormente)
2. Endereço da estrutura que será preenchida com os dados (família de endereços, IP e Porto) do socket conectado
3. Endereço para o tamanho da estrutura definida no ponto 3

Nota: o último parâmetro da função *accept* está à espera de um endereço de memória!

Receber dados

```
#define BUFFERSIZE 4096
int nbytes;
nbytes = recv(sock, buffer, BUFFERSIZE, 0);
buffer[nbytes] = '\0';
```

Diagrama de anotações em vermelho sobre o código:

- 1 sobre `sock`
- 2 sobre `buffer`
- 3 sobre `BUFFERSIZE`
- 4 sobre `0`
- 5 sobre `buffer[nbytes]`

1. Sock (criado anteriormente)
2. Endereço do array de caracteres onde se pretende guardar os dados
3. Tamanho do array de caracteres definido no ponto 2 (**cuidado!** `sizeof()` e não `strlen()`)
4. Flags (não importante por agora)
5. Termina a cadeia de caracteres recebidos com “\0”

Nota: a cadeia de caracteres é terminada com o valor *null* que é representado por “\0” e não “\n”

Enviar dados

```
int nBytesSent;  
int buffer_len = strlen(buffer);  
  
nBytesSent = send(sock, buffer, buffer_len, 0);  
if (nBytesSent == SOCKET_ERROR)  
    fprintf(stderr, "<SER> Impossibilidade de Confirmar.\n");
```

1. Sock (criado anteriormente)
 2. Endereço do array de caracteres que contém os dados que se pretende enviar
 3. Tamanho do array de caracteres definido no ponto 2 (**cuidado! strlen()** e não sizeof())
 4. Flags (não importante por agora)
-

Threads

Permitem dividir o programa em 2 ou + tarefas que serão executadas de forma concorrente (em simultâneo).

Passos para criar threads (ver ficheiro “ComoCriarThreads.c”):

1. Declarar as variáveis (ver formulário)
2. Atribuir os valores à variável “sa” (ver formulário)
3. Criar a thread com a função “CreateThread” (ver formulário + ver imagem/explicação seguinte)
Nota: ter atenção aos parâmetros relativos à função que a thread vai executar e a variável que se quer passar para a função
4. Definir parâmetro como LPVOID (ver formulário)
5. Fazer cast para o tipo de variável que precisamos
6. Terminar thread com ExitThread(...)
Nota: a thread principal (main) também tem que ser terminada da mesma forma.

```

SECURITY_ATTRIBUTES sa;
DWORD thread_id;
int result;

sa.nLength = sizeof(sa);
sa.lpSecurityDescriptor = NULL;

result = CreateThread(1 2, (LPTHREAD_START_ROUTINE)3AtendeCliente, 4(LPVOID)newSock, 5(DWORD)0, 6&thread_id);

if (result == NULL) {
    printf("Nao e' possivel iniciar uma nova thread (error: %d)\n", GetLastError());
    printf("O cliente <%s:%d> nao sera' atendido\n", inet_ntoa(cli_addr.sin_addr), ntohs(cli_addr.sin_port));
    closesocket(newSock);
}

7printf("Sou a thread com id %d\n", GetCurrentThreadId());

```

1. Endereço da estrutura do tipo SECURITY_ATTRIBUTES
 2. Espaço de memória alocado para a thread. Se for 0 (zero), o espaço será o *default*
 3. Função que será executada na thread
 4. Variável que será usada pela função especificada no ponto 3
 5. Flags (não importante por agora)
 6. Endereço para a variável que irá receber o identificador da thread
 7. Retorna o identificar da thread
-

Obter dados do socket

```
int length_addr;  
int result;  
SOCKET sock;  
struct sockaddr_in cli_addr;  
  
length_addr = sizeof(cli_addr);  
  
1      2      3  
result = getpeername(sock, (struct sockaddr*)&cli_addr, &length_addr);  
if (result != SOCKET_ERROR)  
    printf("\nIP = %s, porto = %d\n", inet_ntoa(cli_addr.sin_addr), ntohs(cli_addr.sin_port));
```

1. Socket
 2. Endereço da estrutura que será preenchida
 3. Tamanho da estrutura definida no ponto 2
-

Multiplexagem de sockets

```
int result;  
fd_set fdread;  
struct timeval timeout = { 3, 0 };  
  
result = 1select(232, 3&fdread, 4NULL, 5NULL, &timeout);
```

1. Valor usado para retrocompatibilidade.
Usar o valor que está no formulário
2. Endereço da estrutura que contém o conjunto de sockets
Nota: este conjunto contém os sockets que depois serão verificados se estão prontos para leitura
3. NULL (não importante por agora)
4. NULL (não importante por agora)
5. Endereço da estrutura que contém os valores de timeout
Nota: o primeiro parâmetro da estrutura “timeval” é definido em segundos, o segundo parâmetro é definido em microssegundos

```
1 FD_ZERO(&set);  
2 FD_SET(s, &set);  
3 FD_ISSET(s, &set);
```

1. **FD_ZERO:** Limpa/Inicializa o conjunto de sockets
2. **FD_SET:** Adiciona um socket ao conjunto de sockets
3. **FD_ISSET:** Verifica se um socket faz parte do conjunto de sockets

Passos básicos do fluxo de uma aplicação que usa *select* para gerir vários sockets

```
SOCKET s1;
SOCKET s2;
fd_set fdread;
int result;

while (1) {
    // PASSO 1: Limpar/inicializar o conjunto
    FD_ZERO(&fdread);

    // PASSO 2: Adicionar sockets ao conjunto
    FD_SET(s1, &fdread);
    FD_SET(s2, &fdread);

    // PASSO 3: Executar função select
    // Retorna o número total de sockets que estão prontos para leitura
    result = select(32, &fdread, NULL, NULL, NULL);
    switch (result) {

        case SOCKET_ERROR:
            printf("Erro na rotina select (%d) ...\n", WSAGetLastError());
            closesocket(s1);
            closesocket(s2);
            return;

        default:
            // PASSO 4: Verifica se o foi o s1 que está pronto para leitura
            if (FD_ISSET(s1, &fdread))
            {
                // PASSO 5: faz o que tem a fazer com o socket
            }

            // PASSO 4: Verifica se o foi o s2 que está pronto para leitura
            if (FD_ISSET(s2, &fdread))
            {
                // PASSO 5: faz o que tem a fazer com o socket
            }
            break;
    }
}
```

1. Limpa/Inicializa o conjunto de sockets
 2. Adiciona sockets ao conjunto
 3. Executa a função *select*
 4. Verifica se o socket faz parte do conjunto de sockets
 5. Faz o que tem a fazer com o socket
-

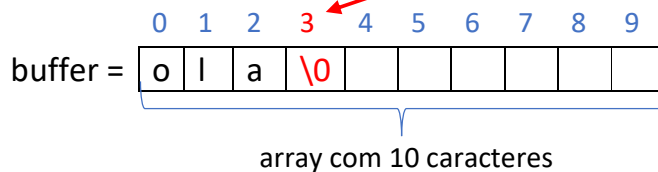
Arrays de caracteres

- Em C não existe o conceito de string mas sim de array de caracteres que são terminados por um caracter nulo → \0
- É por este motivo que se tem **SEMPRE** que terminar a cadeia de caracteres recebidos pela função *recvfrom*
- Se não se terminar a cadeia de caracteres com o “\0”, o resto do array será preenchido com lixo (aqueles 😊 que vimos nas aulas)
- Exemplo de um array de 10 caracteres quando se recebe a mensagem “ola”:

```
// Declarar um array de 10 caracteres
char buffer[10];

// Será recebida a mensagem "ola"
// o que significa que a variável nbytes terá o valor 3 (nº de bytes recebidos)
nbytes = recvfrom(sockfd, buffer, sizeof(buffer), 0, NULL, NULL);
if (nbytes == SOCKET_ERROR)
    Abort("Erro na recepcao de datagrams");

// Aceder à posição 3 (os arrays começam por zero) e colocar o '\0'
// de forma a terminar a cadeia de caracteres
buffer[nbytes]='\0'; // buffer[3] = '\0';
printf("\n<SER1>Mensagem recebida {%s}\n", buffer);
```



Como o número de bytes recebidos é 3 (ola), então estamos a ir à posição 3 do buffer e colocar o “\0”, de modo a terminar a cadeia de caracteres.